

Segment-6

Shortest Path

☐ Single-Source shortest Path:

- It finds the shortest path from a single source vertex to all other vertices in a weighted graph.

SSSP $\begin{cases}$ Dijkstra's Algorithm
Bellman-ford Algorithm

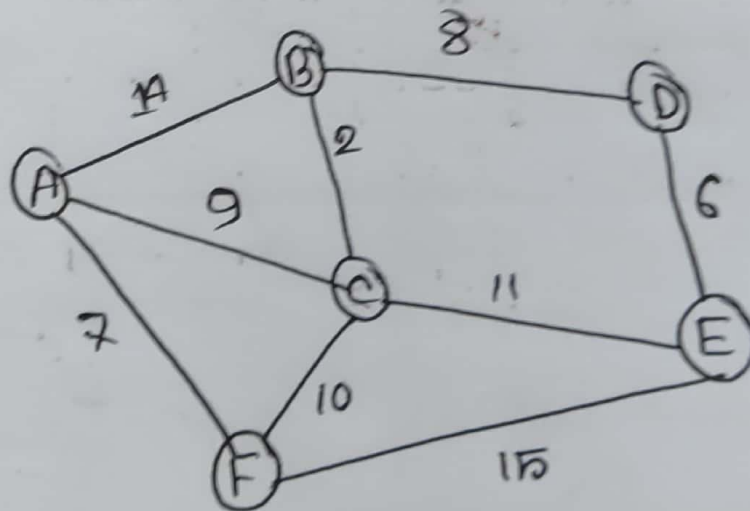
Dijkstra's Algorithm:

Relaxation:

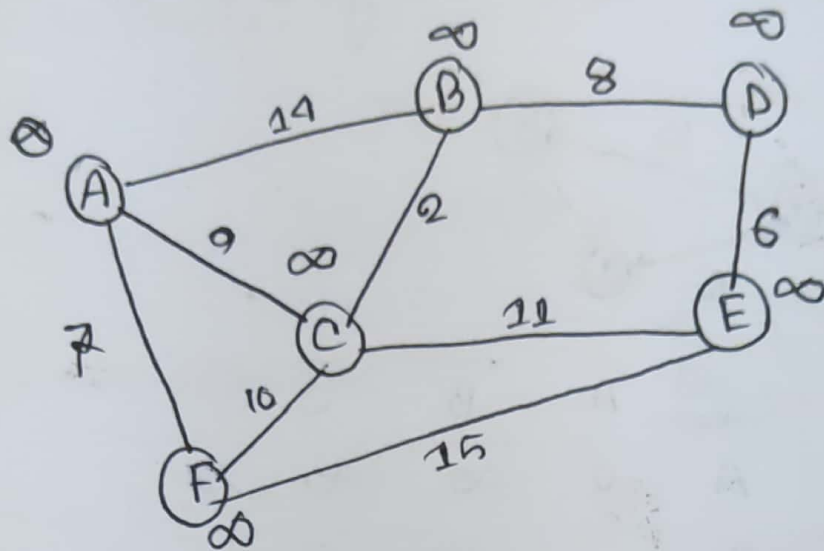
$$\text{if } (d(u) + c(u, v) < d(v)) \\ d(v) = d(u) + c(u, v)$$

$d(u)$ vertex value
 $c(u, v)$ weight

Example:



Step 1: source node ko 0 rakhi aur infinity rakha.



Step 2:

Visited vertex	A	B	C	D	E	F
A	<u>0</u>	∞	∞	∞	∞	∞
F		14	9	∞	∞	<u>7</u>
C		14	<u>9</u>	∞	21	
B		<u>11</u>		∞	20	
D				<u>19</u>	20	
E					<u>20</u>	

vertex value

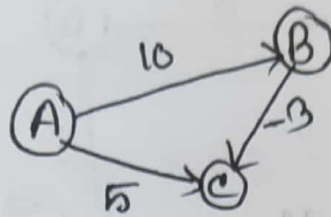
$A \rightarrow C \rightarrow E$

[Backtracking karke minimum value change karke vertex milenge]



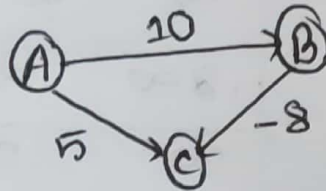
→ Dijkstra's Algorithm doesn't always work on negative weights.

working:



visited vertex	A	B	C
A	0	∞	∞
C		10	<u>5</u>
B		<u>10</u>	

Not working:



visited vertex	A	B	C
A	0	∞	∞
C		∞	<u>5</u>
B			

- Time complexity :

min heap - heap sort $O(n \log n)$

T.c - $O(E \log V)$

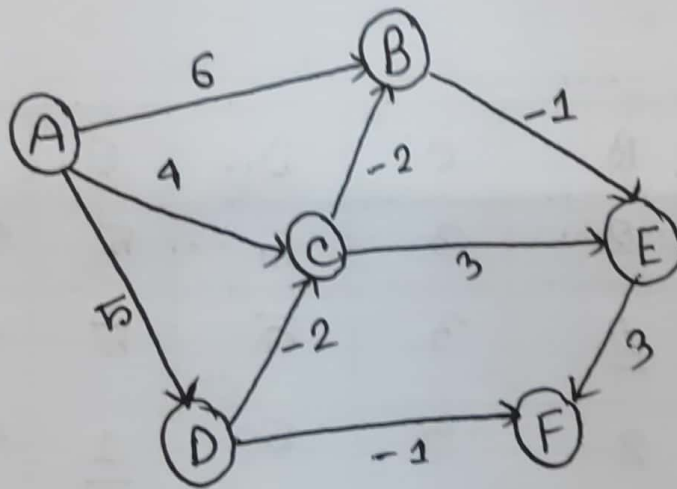
- Dijkstra's Algorithm is a Greedy Algorithm.

Bell-man ford Algorithm :

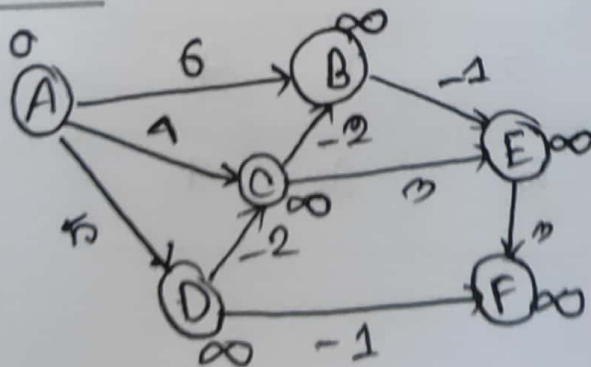
Relaxation : if $(d(u) + c(u, v) < d(v))$
 $d(v) = d(u) + c(u, v)$

- Total Iteration : $(n-1)$ where n is vertex number.

Example :



Step:1 : Source node 0 থেকে বাকিগুলো ∞ ধরে রাখা।



Step-2:

First iteration:

visited vertex	A	B	C	D	E	F
A	<u>0</u>	∞	∞	∞	∞	∞
B	0	<u>6</u>	4	5	∞	∞
C	0	6	<u>4</u>	5	5	∞
D	0	2	4	<u>5</u>	5	∞
E	0	2	3	5	<u>5</u>	4
F	0	2	3	5	5	<u>4</u>

[Relaxation operation]

2nd iteration:

visited vertex	A	B	C	D	E	F
A	<u>0</u>	2	3	5	5	4
B	0	<u>2</u>	3	5	5	4
C	0	2	<u>3</u>	5	<u>1</u>	4
D	0	<u>1</u>	3	<u>5</u>	1	4
E	0	1	3	5	<u>1</u>	4
F	0	1	3	5	1	<u>4</u>

3rd Iteration:

Visited vertex	A	B	C	D	E	F
A	0	1	3	5	1	4
B	0	1	3	5	1	4
C	0	1	3	5	0	4
D	0	1	3	5	0	4
E	0	1	3	5	0	4
F	0	1	3	5	0	3

[3rd iteration - 0th value same as 1st iteration 2(4)]

4th Iteration:

Visited vertex	A	B	C	D	E	F
A	0	1	3	5	0	3
B	0	1	3	5	0	3
C	0	1	3	5	0	3
D	0	1	3	5	0	3
E	0	1	3	5	0	3
F	0	1	3	5	0	3

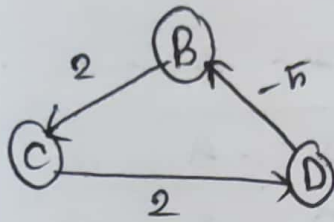
we will stop the iteration.

The answer is:



Drawbacks of Bellman-ford Algorithm:

- Can't work on negative cycle.



1st iteration:

	B	C	D
B	∞	0	∞
C	∞	0	2
D	-3	0	2
	-3	-1	2

2nd iteration:

	B	C	D
B	-3	-1	2
C	-3	-1	2
D	-3	-1	1
	-4	-1	1

For 3rd vertex iteration will be 2. Here, after 2 iteration the value was still changing. So, we can see that bell-fordman algorithm is not applicable in negative cycle.

• Time complexity:

$$O(V^2E)$$

Differences between Dijkstra's Algorithm and Bell-man ford algorithm

Dijkstra's Algorithm	Bell-man ford Algorithm
① Dijkstra's Algorithm may or may not work when there is negative weight edge.	① Bell-man ford's Algorithm works when there is negative weight edge.
② The result contains the vertices containing whole information about the network.	② The result contains the vertices which contains the information about the other vertices they are connected to.
③ It can not be implemented easily in a distributed way.	③ It can no easily be implemented in a distributed way.
④ It is less time consuming.	④ It is more time consuming than Dijkstra's Algorithm.
⑤ It's time complexity is $O(E)$ $O(E \log V)$	⑤ It's time complexity is $O(VE)$
⑥ Greedy approach is taken to implement the algorithm	⑥ Dynamic Programming approach is taken to implement the algorithm



Dijkstra's Algorithm	Bellman ford Algorithm
⑦ Dijkstra's Algorithm have less overheads than Bellman ford's Algorithm.	⑧ Bellman ford Algorithm have more overheads than Dijkstra's Algorithm.

All pair Shortest Path's

• Floyd Warhalls Algorithm's

→ Use a different dynamic programming formulation to solve the all-pairs shortest-paths problem on a directed graph

$$G = (V, E)$$

→ The resulting algorithm, known as the Floyd-Warshall algorithm, runs in $O(V^3)$ time.

• We assume that there are no negative weight cycle.
→ It doesn't works on negative cycle.

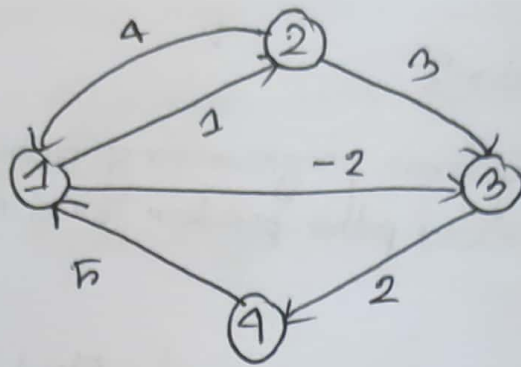
Formula's

$$D_{ij}^k = \begin{cases} w_{ij} & \text{if } k=0 \\ \min(D_{ij}^{k-1}, D_{ik}^{k-1} + D_{kj}^{k-1}) & \text{if } k \geq 1 \end{cases}$$

$$\pi_{ij}^k = \begin{cases} N & \text{if } i=j \text{ or } w_{ij} = \infty \\ i & \text{if } i \neq j \text{ and } w_{ij} < \infty \end{cases}$$



Example:



Soln:

$$D^0 = \begin{bmatrix} 0 & 1 & -2 & \infty \\ 4 & 0 & 3 & \infty \\ \infty & \infty & 0 & 2 \\ 5 & \infty & \infty & 0 \end{bmatrix}$$

$$\pi^0 = \begin{bmatrix} 2 & 1 & 1 & 2 \\ 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 3 \\ 4 & 2 & 2 & 2 \end{bmatrix}$$

$$D^1 = \begin{bmatrix} 0 & 1 & -2 & \infty \\ 4 & 0 & 2 & \infty \\ \infty & \infty & 0 & 2 \\ 5 & 6 & 3 & 0 \end{bmatrix}$$

$$\pi^1 = \begin{bmatrix} 2 & 1 & 1 & 2 \\ 2 & 2 & 1 & 2 \\ 2 & 2 & 2 & 3 \\ 4 & 1 & 1 & 2 \end{bmatrix}$$

$$D^2 = \begin{bmatrix} 0 & 1 & -2 & \infty \\ 4 & 0 & 2 & \infty \\ \infty & \infty & 0 & 2 \\ 5 & 6 & 3 & 0 \end{bmatrix}$$

$$\pi^2 = \begin{bmatrix} 2 & 1 & 1 & 2 \\ 2 & 2 & 1 & 2 \\ 2 & 2 & 2 & 3 \\ 4 & 1 & 1 & 2 \end{bmatrix}$$

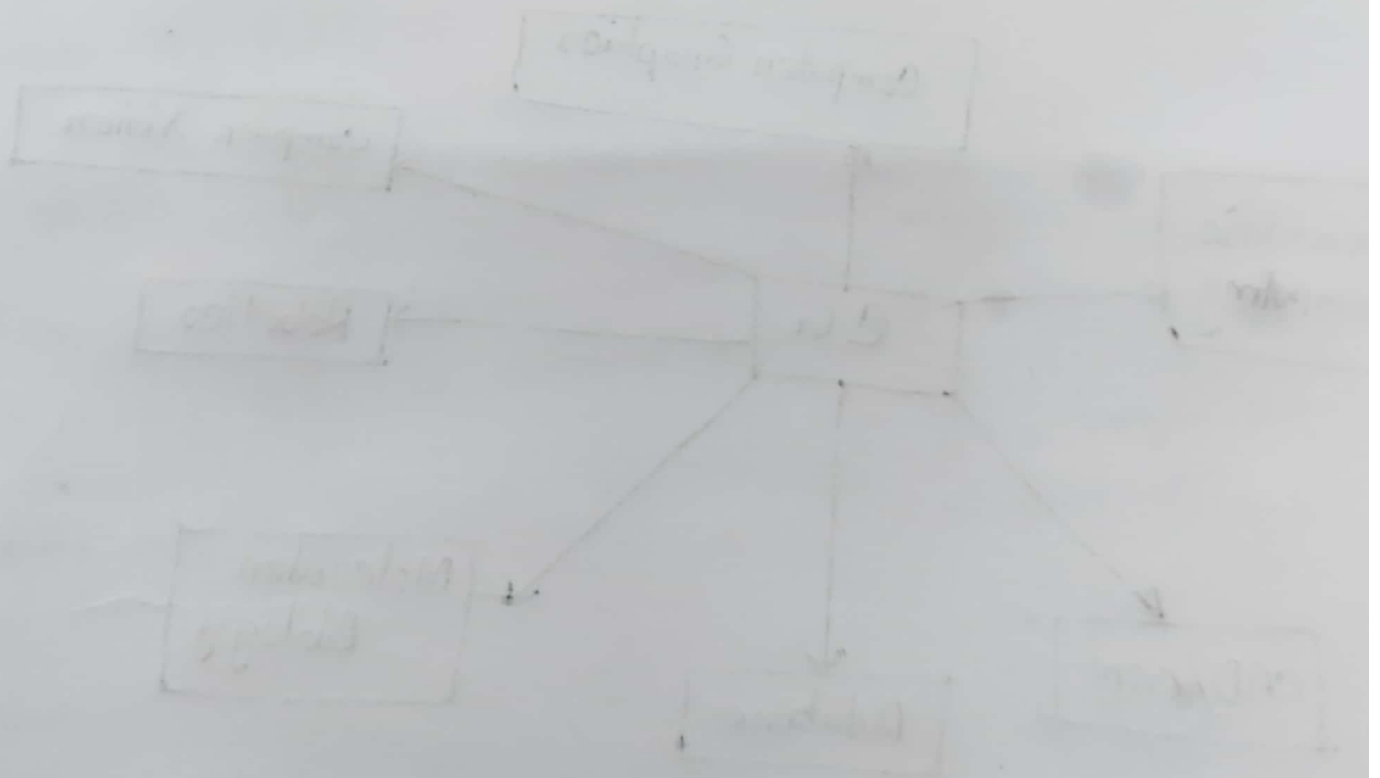
$$D^3 = \begin{bmatrix} 0 & 1 & -2 & 0 \\ 4 & 0 & 2 & 4 \\ \infty & \infty & 0 & 2 \\ 5 & 6 & 3 & 0 \end{bmatrix}$$

$$\pi^3 = \begin{bmatrix} 2 & 1 & 1 & 2 \\ 2 & 2 & 1 & 3 \\ 2 & 2 & 2 & 3 \\ 4 & 1 & 1 & 2 \end{bmatrix}$$

$$D^A = \begin{bmatrix} 0 & 1 & -2 & 0 \\ 4 & 0 & 2 & 4 \\ 7 & 8 & 0 & 2 \\ 5 & 6 & 3 & 0 \end{bmatrix}$$

$$\pi^A = \begin{bmatrix} N & 1 & 1 & N \\ 2 & N & 1 & 3 \\ 4 & 4 & N & 3 \\ 4 & 1 & 1 & N \end{bmatrix}$$

Am



MigTM
Mirogabalin Besylate INN
Film Coated Tablet

25 mg
5 mg
10 mg